Relajación Bidireccional In-Place y Poda por Dominancia Geométrica para Caminos Mínimos en Digrafos Planares Triangulares

Lucio Romero Mascaró

Investigador Independiente

lucioromeromascaro@gmail.com | github.com/lucio-rm

17 de octubre de 2026

Resumen

Se aborda la resolución exacta del problema del camino de costo mínimo desde la cima hasta la base en digrafos planares triangulares estratificados de n capas ($N = |V| = \frac{n(n+1)}{2}$ vértices) con pesos estrictamente positivos en los vértices ($w : V \to \mathbb{R}^+$).

El *Obstáculo* fue la admisión de transiciones horizontales bidireccionales dentro de una misma capa induce ciclos dirigidos de longitud 2. Esta topología destruye el ordenamiento topológico exigido por la Programación Dinámica y fuerza a el algoritmo de Dijkstra mediante heaps (colas de prioridad) en tiempo $\mathcal{O}(|V| \log |V|) = \mathcal{O}(n^2 \log n)$.

El método para *resolver* el problema fue usar una descomposición direccional de estados en dos barridos monótonos sucesivos (derecha-izquierda para capturar transiciones no crecientes en columna, e izquierda-derecha para consolidar transiciones crecientes) ejecutados *in-place* sobre una única estructura escalonada, acoplada con un teorema de poda algebraica por dominancia geométrica en las fronteras del triángulo.

Resultado: Se demuestra formalmente la obtención del óptimo global en tiempo y espacio estrictamente lineales respecto al tamaño de la entrada, $\Theta(N) = \Theta(n^2)$, eliminando el factor logarítmico de Dijkstra y reduciendo en un 75 % las reservas de memoria frente a implementaciones matriciales estándar.

Índice

Resumen **1**

1. Introducción y Motivación **3**

1.1. Impacto Interdisciplinario e Isomorfismo Estructural 3

2. Formulación Matemática del Dominio **4**

2.1. Análisis del Ciclo: El Fracaso del Planteo Ingenuo 5

3. Diseño del Algoritmo: Relajación Bidireccional y Poda **6**

3.1. Descomposición Bidireccional por Monotonía Horizontal 6

3.2. Teorema de Poda por Dominancia Geométrica en las Fronteras 6

3.3. Optimización Espacial In-Place e Implementación 8

4. Análisis del Algoritmo **9**

4.1. Demostración de Optimalidad Global 9

4.2. Análisis de Complejidad Temporal y Espacial 10

4.2.1. Impacto de la Variabilidad de los Datos 11

5. Evaluación Experimental y Comparativa Empírica **12**

5.1. Ajuste por Cuadrados Mínimos de la Programación Dinámica 12

5.2. Comparativa Empírica: Relajación Bidireccional vs. Dijkstra 12

6. Conclusión y Líneas Futuras **13**

Bibliografía **14**

1. Introducción y Motivación

El cálculo de caminos de costo mínimo sobre grafos ponderados constituye uno de los problemas centrales de la ciencia de la computación y la investigación operativa (Cormen et al. 2022; Kleinberg y Tardos 2005). Cuando un grafo dirigido carece de ciclos conformando un Grafo Dirigido Acíclico (DAG), es posible establecer un ordenamiento topológico sobre sus vértices y resolver el camino mínimo mediante Programación Dinámica en tiempo lineal $\Theta(|V| + |E|)$, evaluando cada estado exactamente una vez a través de la ecuación de optimalidad de Bellman (Bellman 1957). Por el contrario, cuando el grafo presenta ciclos dirigidos con pesos positivos, la ausencia de un orden topológico impide la tabulación directa. La resolución estándar recae entonces en el algoritmo greedy de Dijkstra (Dijkstra 1959), cuya dependencia de un *heap* (cola de prioridad) impone un sobrecosto asintótico de $\mathcal{O}(|E| + |V| \log |V|)$.

En el presente trabajo se estudia una estructura topológica híbrida: una pirámide o grilla triangular de n niveles donde las transiciones entre capas sucesivas $(i-1 \to i)$ son estrictamente dirigidas hacia abajo, pero las transiciones entre capas permiten movimientos laterales en ambos sentidos $((i, j) \leftrightarrow (i, j+1))$.

Si bien desde una perspectiva intuitiva ningún camino óptimo con pesos positivos retrocedería sobre sus propios pasos (ya que sumar pesos positivos adicionales es subóptimo), una formulación directa de Programación Dinámica *fracasa*: el estado (i, j) requiere conocer el óptimo de $(i, j+1)$, el cual a su vez requiere conocer el óptimo de (i, j), generando una dependencia circular mutua (Buchwald y Genender 2026).

Como se ilustra en la Figura 1, demostramos que es posible desacoplar estos componentes cíclicos entre capas *sin* recurrir a Dijkstra, combinando una relajación bidireccional en dos fases con una poda algebraica sobre los vértices de las esquinas (marcados en rojo punteado), cuyas transiciones hacia el interior resultan geométricamente dominadas.

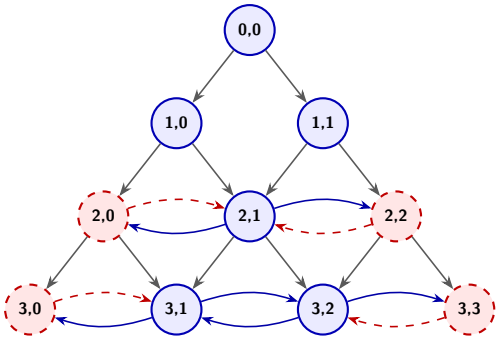


Figura 1: Digrafo triangular con ciclos horizontales y poda de frontera (en rojo).

1.1. Impacto Interdisciplinario e Isomorfismo Estructural

La resolución en tiempo lineal estricto $\Theta(N)$ de grillas estratificadas con desvíos laterales trasciende el ámbito académico y presenta isomorfismos directos con problemas críticos en cuatro disciplinas:

- **Visión por Computadora (*Piecewise-Horizontal Seam Carving*):** El algoritmo clásico de redimensionamiento inteligente de imágenes de Avidan y Shamir (2007) busca "costuras" verticales de mínima energía mediante Programación Dinámica descendente $((i-1, j-1), (i-1, j), (i-1, j+1) \to (i, j))$, prohibiendo desplazamientos horizontales en la misma fila para evitar ciclos. Cuando una imagen contiene objetos anchos que exigen desviar la costura

lateralmente dentro de una misma fila de píxeles, el esquema de doble barrido presentado en este informe permite incorporar aristas horizontales manteniendo la complejidad lineal por píxel, vital para procesamiento de video en tiempo real.

- **Diagnóstico Médico por Imágenes (Segmentación Retiniana en OCT):** En oftalmología, la Tomografía de Coherencia Óptica (SD-OCT) requiere segmentar automáticamente las capas celulares de la retina (Chiu et al. 2010). La tomografía se modela como un digrafo por capas donde los pesos reflejan gradientes de intensidad tisular. En pacientes con edemas maculares o drusas, las capas celulares sufren pliegues abruptos que obligan al contorno óptimo a desplazarse lateralmente dentro del mismo estrato. Reemplazar el algoritmo de Dijkstra ($\mathcal{O}(N \log N)$) por mi relajación bidireccional de ($\Theta(N)$) elimina el cuello de botella del *heap* durante la reconstrucción volumétrica 3D en quirófano.
- **Economía y Finanzas Cuantitativas (Árboles Binomiales con Rebalanceo):** El modelo común de valuación de derivados financieros de Cox, Ross y Rubinstein (1979) estructura la evolución temporal del precio de un activo como un árbol binomial recombinante donde el período i posee exactamente $i + 1$ estados posibles, conformando un triángulo isomorfo al analizado. Si un inversor puede rebalancear su cobertura de riesgo dentro de un mismo período pagando un costo de transacción positivo ($w > 0$), el Teorema de Poda de Frontera demuestra analíticamente que en los estados extremos del mercado ($j = 0$ y $j = i$) jamás es racional llegar a un estado interior rebotando contra los límites de colapso o burbuja máxima.
- **Teoría Algorítmica de Juegos (Juegos Secuenciales con Renegociación):** En juegos finitos extensivos por etapas donde el estado (i, j) representa la distribución de recursos en la ronda i, las aristas laterales modelan instancias de renegociación horizontal con peaje positivo antes de avanzar a la etapa $i + 1$. La poda demostrada garantiza que las configuraciones extremas carecen de poder de transición hacia el interior del subjuego.

2. Formulación Matemática del Dominio

Defino formalmente la pirámide de n filas como un digrafo planar estratificado y ponderado en sus vértices $G = (V, E, w)$, compuesto por los siguientes elementos:

1. **Conjunto de Vértices (V):** El dominio espacial se organiza en n capas discretas indexadas por $i \in \{0, 1, \ldots, n-1\}$, donde cada capa i contiene exactamente $i + 1$ vértices indexados por columna $j \in \{0, 1, \ldots, i\}$:

$$V = \{(i, j) \in \mathbb{Z}^2 \mid 0 \leq i < n,\ 0 \leq j \leq i\}$$

   La cardinalidad total del grafo (que define el tamaño real de la entrada N) responde a la serie aritmética de Gauss:

$$N = |V| = \sum_{i=0}^{n-1} (i + 1) = \frac{n(n+1)}{2} = \Theta(n^2)$$

2. **Conjunto de Aristas Dirigidas (E):** El conjunto de transiciones válidas se particiona en dos subconjuntos disjuntos, $E = E_{\text{down}} \cup E_{\text{horiz}}$:

- *Aristas descendentes entre capas (E_{down}):* Conectan un vértice superior con sus dos hijos inmediatos en la capa inferior, impidiendo cualquier retroceso hacia filas superiores:

$$E_{\text{down}} = \{((i-1, j-1) \to (i, j)) \mid 0 < j \leq i\} \cup \{((i-1, j) \to (i, j)) \mid 0 \leq j < i\}$$

- *Aristas bidireccionales entre capas (E_{horiz}):* Conectan vértices adyacentes dentro de un mismo nivel i:

$$E_{\text{horiz}} = \{((i, j) \to (i, j+1)) \mid 0 \leq j < i\} \cup \{((i, j+1) \to (i, j)) \mid 0 \leq j < i\}$$

3. **Función de Pesos Positivos (w):** A cada vértice $(i, j) \in V$ se le asigna un costo escalar estrictamente positivo $v_{i,j} = w(i, j) \in \mathbb{R}^+$ (o $\mathbb{Z}^+$). El objetivo consiste en hallar una trayectoria dirigida $P = \langle (0, 0), \ldots, (n-1, j^*) \rangle$ desde la cima hasta cualquier vértice de la base que minimice el funcional de costo acumulado $C(P) = \sum_{(i,j) \in P} v_{i,j}$.

Necesidad teórica de la restricción $v_{i,j} > 0$ (Ausencia de Ciclos Negativos):
La hipótesis de positividad estricta resulta indispensable para la buena definición del problema. Si se admitieran pesos negativos ($v_{i,j} < 0$), cualquier par de vértices adyacentes en una misma fila tal que $v_{i,j} + v_{i,j+1} < 0$ (por ejemplo, $v_{1,0} = -5$ y $v_{1,1} = 2$, cuya suma cíclica es -3) conformaría un ciclo dirigido de peso negativo alcanzable desde $(0, 0)$. En dicho escenario, alternar indefinidamente $(1, 0) \leftrightarrow (1, 1)$ haría diverger el costo óptimo a $-\infty$. Si por el contrario se exigiera hallar un *camino simple* (sin repetir vértices) en presencia de ciclos negativos, el problema se reduciría al problema del Camino Más Largo, convirtiéndose en **NP-Hard** (Cormen et al. 2022).

2.1. Análisis del Ciclo: El Fracaso del Planteo Ingenuo

Si se intenta formular una ecuación de recurrencia directa considerando simultáneamente las cuatro direcciones de arribo posibles hacia el estado (i, j), se obtiene el siguiente planteo ingenuo:

$$\mathcal{OPT}[i][j] = v_{i,j} + \text{mín} \begin{cases} \mathcal{OPT}[i-1][j-1] & \text{si } i > 0 \wedge j > 0 \quad \text{(arriba-izquierda)} \\ \mathcal{OPT}[i-1][j] & \text{si } i > 0 \wedge j < i \quad \text{(arriba-derecha)} \\ \mathcal{OPT}[i][j-1] & \text{si } j > 0 \quad \text{(izquierda)} \\ \mathcal{OPT}[i][j+1] & \text{si } j < i \quad \text{(derecha)} \end{cases}$$

Demostración de la Invalidez Topológica:
Al evaluar la fila i de izquierda a derecha ($j = 0 \ldots i$), en el instante en que se intenta computar el estado $\mathcal{OPT}[i][j]$, los estados de la capa superior $\mathcal{OPT}[i-1][\cdot]$ ya se encuentran resueltos. Sin embargo, el término $\mathcal{OPT}[i][j+1]$ aún no ha sido calculado (contiene un valor centinela ∞ o 0). Simétricamente, para calcular $\mathcal{OPT}[i][j+1]$ la ecuación exige conocer $\mathcal{OPT}[i][j]$. Esto establece una relación de dependencia circular $\mathcal{OPT}[i][j] \leftrightarrows \mathcal{OPT}[i][j+1]$ que viola la propiedad de aciclicidad de Bellman, haciendo que un único barrido unidireccional omita caminos óptimos que descienden por la derecha y luego se desplazan hacia la izquierda.

3. Diseño del Algoritmo: Relajación Bidireccional y Poda

Para romper la dependencia circular sin alterar la optimalidad global, analizamos la estructura geométrica de cualquier camino óptimo dentro de una misma capa i.

3.1. Descomposición Bidireccional por Monotonía Horizontal

Lema 1 (Monotonía Horizontal entre capas): *En un digrafo triangular estratificado con pesos positivos $v_{i,j} > 0$, ningún camino óptimo puede cambiar de dirección horizontal dentro de una misma fila i.*

Demostración (Por reducción al absurdo):
Supongamos que existe un camino óptimo P^* que, encontrándose en la fila i, realiza un movimiento hacia la derecha $(i, j) \to (i, j + 1)$ e inmediatamente (o tras varios pasos horizontales) invierte su sentido hacia la izquierda regresando a (i, j). Dicho subcamino tendría la forma $\langle \dots, (i, j), \dots, (i, j + k), \dots, (i, j), \dots \rangle$ con $k \geq 1$. Dado que $v_{i,m} > 0$ para todo vértice, si recortamos el ciclo intermedio reemplazando todo el bucle por una única visita al nodo (i, j), obtenemos un camino válido P' cuyo costo verifica:

$$C(P') = C(P^*) - \sum_{\text{ciclo}} v_{i,m} < C(P^*)$$

Esto contradice la hipótesis de que P^* era óptimo. El razonamiento es idéntico si primero se desplaza hacia la izquierda y luego hacia la derecha. ■

Corolario (Descomposición en Dos Fases Acíclicas):
Todo segmento horizontal de un camino óptimo en la fila i es estrictamente monótono: o bien ingresa a la fila i por una columna $k \geq j$ y se desplaza exclusivamente hacia la izquierda ($\leftarrow$), o bien ingresa por una columna $k \leq j$ y se desplaza exclusivamente hacia la derecha ($\rightarrow$). Esto permite descomponer el cálculo de la fila i en dos subgrafos acíclicos independientes:

1. **Estado Auxiliar Estrella ($\mathcal{OPT}^*[i][j]$):** Representa el costo mínimo para llegar a (i, j) permitiendo únicamente descensos desde la fila $i - 1$ y movimientos horizontales *hacia la izquierda*. Se calcula yendo de la fila de derecha a izquierda.
2. **Estado Definitivo ($\mathcal{OPT}[i][j]$):** Consolida el costo mínimo absoluto comparando el mejor camino proveniente de la derecha ($\mathcal{OPT}^*[i][j]$) contra la opción de venir desde el vecino izquierdo ya consolidado ($\mathcal{OPT}[i][j-1] + v_{i,j}$). Se calcula yendo de izquierda a derecha.

3.2. Teorema de Poda por Dominancia Geométrica en las Fronteras

A diferencia de una grilla rectangular donde el doble barrido debe evaluar los m elementos completos en ambas direcciones, demostramos que la geometría triangular induce una relación de dominancia estricta en ambas esquinas de cada fila $i \geq 2$.

Teorema 1 (Poda de Esquinas y Reducción de Estados Estrella): *Para toda capa $i \geq 2$, los vértices extremos $p = 0$ (esquina izquierda) y $u = i$ (esquina derecha) nunca pueden formar parte de un camino óptimo que se desplace horizontalmente hacia el interior de la fila i.*

En consecuencia: (a) el pre-cálculo de $\mathcal{OPT}^*$ en el anteúltimo elemento $a = i-1$ prescinde de evaluar a $u = i$, y (b) el segundo elemento $d = 1$ verifica $\mathcal{OPT}[i][1] = \mathcal{OPT}^*[i][1]$ sin evaluar a $p = 0$.

Demostración Algebraica:

- **Parte (a) — Dominancia en la Frontera Derecha ($u = i$ y $a = i-1$):**
  Por la topología triangular, el extremo derecho (i, i) carece de padre superior derecho, la fila $i-1$ solo llega hasta la columna $i-1$. Su único padre en la capa superior es $(i-1, i-1)$. Si un camino desciende desde $(i-1, i-1)$ hacia la esquina (i, i) y luego retrocede horizontalmente hacia el anteúltimo nodo $(i, i-1)$, su costo acumulado es:

  $$C_{\text{esquina}} = \mathcal{OPT}[i-1][i-1] + v_{i,i} + v_{i,i-1}$$

  Sin embargo, el vértice $(i-1, i-1)$ es simultáneamente padre directo de $(i, i-1)$ mediante una arista descendente. El costo de bajar directamente de $(i-1, i-1)$ a $(i, i-1)$ es:

  $$C_{\text{directo}} = \mathcal{OPT}[i-1][i-1] + v_{i,i-1}$$

  Como $v_{i,i} > 0$, se cumple invariablemente que $C_{\text{directo}} < C_{\text{esquina}}$. Por lo tanto, la transición $(i, i) \to (i, i-1)$ es estrictamente subóptima y se poda del cálculo, definiendo el caso base derecho de $\mathcal{OPT}^*$ como:

  $$\mathcal{OPT}^*[i][a] = v_{i,a} + \min\left(\mathcal{OPT}[i-1][a-1],\ \mathcal{OPT}[i-1][a]\right)$$

- **Parte (b) — Dominancia en la Frontera Izquierda ($p = 0$ y $d = 1$):**
  De manera simétrica, el extremo izquierdo $(i, 0)$ tiene un único padre superior: $(i-1, 0)$. Cualquier camino que descienda a $(i, 0)$ y luego avance horizontalmente hacia la derecha al segundo nodo $(i, 1)$ da un costo $\mathcal{OPT}[i-1][0] + v_{i,0} + v_{i,1}$. Dado que $(i-1, 0)$ también conecta directamente con $(i, 1)$ con costo $\mathcal{OPT}[i-1][0]+v_{i,1}$ (el cual ya fue evaluado dentro de $\mathcal{OPT}^*[i][1]$), y siendo $v_{i,0} > 0$, el paso desde $(i, 0)$ hacia $(i, 1)$ queda estrictamente dominado. En consecuencia, el segundo elemento consolida su óptimo directamente sin comparaciones adicionales:

  $$\mathcal{OPT}[i][1] = \mathcal{OPT}^*[i][1]$$

Gracias a las dos podas, para una fila de $k = i+1$ elementos solo se requieren calcular $k-2 = i-1$ estados estrella (desde $j = i-1$ hasta $j = 1$). ■

Resumen de la Ecuación de Recurrencia Final (Para toda capa $i \geq 2$):
Definiendo los punteros notables $p = 0$, $d = 1$, $u = i$ y $a = i-1$:

Fase 1 ($\mathcal{OPT}^*$):
$$\begin{cases} \mathcal{OPT}^*[i][a] = v_{i,a} + \min\left(\mathcal{OPT}[i-1][a-1],\ \mathcal{OPT}[i-1][a]\right) \\ \mathcal{OPT}^*[i][j] = v_{i,j} + \min\left(\mathcal{OPT}[i-1][j-1],\ \mathcal{OPT}[i-1][j],\ \mathcal{OPT}^*[i][j+1]\right) \quad \forall j \in [a-1 \dots 1] \end{cases}$$

Fase 2 ($\mathcal{OPT}$):
$$\begin{cases} \mathcal{OPT}[i][p] = v_{i,p} + \min\left(\mathcal{OPT}[i-1][p],\ \mathcal{OPT}^*[i][d]\right) \\ \mathcal{OPT}[i][d] = \mathcal{OPT}^*[i][d] \\ \mathcal{OPT}[i][j] = \min\left(v_{i,j} + \mathcal{OPT}[i][j-1],\ \mathcal{OPT}^*[i][j]\right) & \forall j \in [2 \dots u-1] \\ \mathcal{OPT}[i][u] = v_{i,u} + \min\left(\mathcal{OPT}[i-1][u-1],\ \mathcal{OPT}[i][u-1]\right) \end{cases}$$

3.3. Optimización Espacial In-Place e Implementación

En una primera aproximación podrían instanciarse dos matrices cuadradas de $n \times n$ (`ESTRELLA` y `OPT`), consumiendo $2n^2$ celdas en memoria. Sin embargo, observamos dos propiedades clave que permiten reducir drásticamente el consumo espacial:

1. **Dimensionamiento Escalonado:** La fila i solo posee $i+1$ columnas válidas. Al reservar listas de longitud exacta $i+1$, reducimos el espacio base de n^2 a $\frac{n(n+1)}{2} \approx 0.5n^2$.
2. **Sobrescritura In-Place (Eliminación total de `ESTRELLA`):** Los valores de $\mathcal{OPT}^*[i]$ solo se utilizan durante el cálculo de la misma fila i. Por lo tanto, en la Fase 1 escribimos los valores de $\mathcal{OPT}^*$ directamente dentro del arreglo definitivo `OPT[i]` de derecha a izquierda. Después, en la Fase 2, recorremos `OPT[i]` de izquierda a derecha actualizando `OPT[i][j] = min(OPT[i][j], piramide[i][j] + OPT[i][j-1])`, donde el valor previo que estaba en `OPT[i][j]` es exactamente $\mathcal{OPT}^*[i][j]$. Esto elimina el 100 % de la matriz auxiliar `ESTRELLA`, logrando un ahorro total del 75 % de memoria respecto al planteo matricial ingenuo.

A continuación se presenta la implementación completa en lenguaje Python:

```python
def camino_minimo_piramide(piramide: list[list[int]]) -> tuple[int, list[list[int]]]:
    if not piramide:
        return 0, []
    n = len(piramide)
    OPT = [[0] * (i + 1) for i in range(n)]  # Matriz escalonada unica (sin tabla ESTRELLA)

    # Casos base (filas 0 y 1)
    OPT[0][0] = piramide[0][0]
    if n == 1:
        return OPT[0][0], [[0, 0]]
    OPT[1][0] = piramide[1][0] + OPT[0][0]
    OPT[1][1] = piramide[1][1] + OPT[0][0]

    for i in range(2, n):
        p, d, u, a = 0, 1, i, i - 1

        # Fase 1: Barrido Derecha a Izquierda (OPT* almacenado in-place sobre OPT[i])
        OPT[i][a] = piramide[i][a] + min(OPT[i - 1][a - 1], OPT[i - 1][a])
        for j in range(a - 1, 0, -1):
```

```
            OPT[i][j] = piramide[i][j] + min(OPT[i - 1][j - 1], OPT[i - 1][
    j], OPT[i][j + 1])

        # Fase 2: Barrido Izquierda a Derecha (Consolidacion de OPT
    definitivo)
        OPT[i][p] = piramide[i][p] + min(OPT[i - 1][p], OPT[i][d])
        # OPT[i][d] ya contiene OPT*[i][d], queda consolidado por Teorema
    1(b)
        for j in range(2, u):
            OPT[i][j] = min(OPT[i][j], piramide[i][j] + OPT[i][j - 1])
        OPT[i][u] = piramide[i][u] + min(OPT[i - 1][u - 1], OPT[i][u - 1])

    ult = n - 1
    min_valor = min(OPT[ult])
    min_col = OPT[ult].index(min_valor)
    return min_valor, _reconstruccion(piramide, OPT, [min_valor, ult,
    min_col])

def _reconstruccion(piramide: list[list[int]], OPT: list[list[int]],
    destino: list) -> list[list[int]]:
    if destino[0] == float('inf'):
        return []
    camino = []
    fila, col = destino[1], destino[2]
    camino.append([fila, col])

    while fila > 0:
        buscado = OPT[fila][col] - piramide[fila][col]
        if col > 0 and OPT[fila - 1][col - 1] == buscado:
            fila, col = fila - 1, col - 1
        elif col < fila and OPT[fila - 1][col] == buscado:
            fila -= 1
        elif col > 0 and OPT[fila][col - 1] == buscado:
            col -= 1
        elif col < fila and OPT[fila][col + 1] == buscado:
            col += 1
        camino.append([fila, col])

    return camino[::-1]
```

Listing 1: Implementación del algoritmo de PD Bidireccional In-Place y su reconstrucción.

4. Análisis del Algoritmo

4.1. Demostración de Optimalidad Global

Demuestro mediante el **Principio de Inducción Matemática** sobre el índice de capa i que, al finalizar la ejecución de la Fase 2 en el nivel i, toda celda $\mathcal{OPT}[i][j]$ almacena el costo mínimo

exacto desde $(0,0)$ hasta (i,j).

- **Casos Base ($i = 0$ e $i = 1$):** Para $i = 0$, el único estado es el origen $(0,0)$ con costo $\mathcal{OPT}[0][0] = v_{0,0}$. Para $i = 1$, por el Teorema 1, bajar directamente de $(0,0)$ a $(1,0)$ cuesta $v_{0,0} + v_{1,0}$, lo cual domina estrictamente al desvío $(0,0) \to (1,1) \to (1,0)$ de costo $v_{0,0} + v_{1,1} + v_{1,0}$. Simétricamente ocurre para $(1,1)$. Por ende, las filas 0 y 1 son óptimas.
- **Hipótesis Inductiva $P(i-1)$:** Asumimos que para la fila $i-1$ todos los estados $\mathcal{OPT}[i-1][k]$ $(0 \le k \le i-1)$ contienen el costo mínimo global hacia $(i-1,k)$.
- **Paso Inductivo $P(i)$:** Sea P^* el camino óptimo hacia (i,j). Por definición del digrafo, P^* debe ingresar por primera vez a la fila i descendiendo desde algún padre en la fila $i-1$ hacia alguna columna de entrada $k \in \{0,\dots,i\}$, y luego desplazarse horizontalmente desde (i,k) hasta (i,j). Por el Lema 1 (Monotonía Horizontal), existen solo dos casos mutuamente excluyentes:

  1. *Caso $k \ge j$ (Ingreso por la derecha o directo):* El camino baja a (i,k) y se mueve exclusivamente hacia la izquierda hasta (i,j). Como la Fase 1 barre de derecha a izquierda evaluando en cada paso mín$(\mathcal{OPT}[i-1][\text{padres}], \mathcal{OPT}^*[i][\text{derecha}])$ y los padres en $i-1$ son óptimos por Hipótesis Inductiva, por inducción interna este camino es capturado exactamente por $\mathcal{OPT}^*[i][j]$ (exceptuando $k = i$ cuando $j \le i-1$, ya descartado por el Teorema 1).
  2. *Caso $k < j$ (Ingreso por la izquierda):* El camino baja a (i,k) y se mueve exclusivamente hacia la derecha pasando obligatoriamente por $(i,j-1)$ antes de pisar (i,j). Como la Fase 2 barre de izquierda a derecha, al evaluar j el estado $\mathcal{OPT}[i][j-1]$ ya es óptimo y la transición $v_{i,j} + \mathcal{OPT}[i][j-1]$ captura este subcamino.

  Dado que la Fase 2 toma el mínimo entre ambos casos exhaustivos, $\mathcal{OPT}[i][j]$ es óptimo para todo j. Al tomar finalmente mín$_j(\mathcal{OPT}[n-1][j])$ en la base, el algoritmo garantiza el óptimo global. ■

4.2. Análisis de Complejidad Temporal y Espacial

Cumpliendo con el rigor metodológico de separar la etapa de tabulación de la etapa de reconstrucción, analizamos ambas complejidades en función de la altura n y del tamaño de entrada $N = |V| = \frac{n(n+1)}{2}$.

1. Complejidad del Llenado de la Tabla de Óptimos de Programación Dinámica:

- **Complejidad Temporal:** Para cada fila $i \in [2, n-1]$, el primer bucle interno (Fase 1) realiza $i-2$ iteraciones de costo $\mathcal{O}(1)$, y el segundo bucle interno (Fase 2) realiza $i-2$ iteraciones de costo $\mathcal{O}(1)$, sumado a 4 asignaciones constantes de borde. El trabajo total es:

$$T_{\text{PD}}(n) = \mathcal{O}(1) + \sum_{i=2}^{n-1} \big(2(i-2)+4\big) \cdot \mathcal{O}(1) = \sum_{i=2}^{n-1} \mathcal{O}(i) = \Theta(n^2) = \Theta(N)$$

  Dado que cualquier algoritmo exacto debe inspeccionar al menos una vez cada uno de los $N = \Theta(n^2)$ números de la entrada (estableciendo una cota inferior teórica $\Omega(N) = \Omega(n^2)$), mi

algoritmo alcanza la **optimalidad asintótica estricta**: es cuadrático respecto a la cantidad de filas n, pero **lineal** $\Theta(N)$ **respecto al tamaño de la entrada**.

- **Complejidad Espacial y Refutación del Mito $\mathcal{O}(n)$:** La estructura escalonada `OPT` aloja exactamente $\sum_{i=0}^{n-1}(i+1) = \frac{n(n+1)}{2}$ enteros, por lo que la complejidad espacial es $\Theta(n^2) = \Theta(N)$. Cabe destacar que si el problema solo solicitara devolver el *valor escalar* del camino mínimo, bastaría con conservar en memoria únicamente la fila anterior `OPT[i-1]` para computar `OPT[i]`, reduciendo el espacio a $\mathcal{O}(n)$. Sin embargo, dado que se exige **reconstruir el camino completo**, descartar las filas superiores destruiría la historia de estados previa, haciendo imposible rastrear las decisiones hacia atrás sin recurrir a técnicas de recomputación recursiva del tipo Hirschberg (1975).

2. Complejidad de la Reconstrucción y el Caso tipo "Serpiente":
En una pirámide tradicional donde solo se permite descender, el camino reconstruido disminuye la variable `fila` en cada iteración, tardando siempre $\Theta(n)$. En este problema, la presencia de movimientos laterales introduce una diferencia fundamental entre el mejor y el peor caso:

- **Mejor Caso Temporal $\Omega(n)$:** El camino óptimo desciende directamente desde $(0,0)$ hasta la base sin realizar movimientos horizontales, dando exactamente n pasos en el bucle `while`.
- **Peor Caso Temporal (Caso tipo "Serpiente") $\Theta(n^2) = \Theta(N)$:** Considerar una pirámide donde todas las celdas tienen peso 999, excepto un corredor en forma de "S" (zig-zag) con peso 1 que baja por la derecha a la fila 2, recorre toda la fila 2 hacia la izquierda, baja a la fila 3, recorre la fila 3 hacia la derecha, y así sucesivamente:

$$\text{Pirámide Serpiente} = \begin{bmatrix} \mathbf{1} & & & & \\ 999 & \mathbf{1} & & & \\ \mathbf{1} & \mathbf{1} & \mathbf{1} & & \\ \mathbf{1} & 999 & 999 & 999 & \\ \mathbf{1} & \mathbf{1} & \mathbf{1} & \mathbf{1} & 999 \end{bmatrix}$$

En este escenario patológico, el camino óptimo atraviesa horizontalmente casi todos los elementos de cada fila i (hasta i celdas por capa por el Teorema 1). En consecuencia, el bucle `while` de `_reconstruccion` realiza $\sum_{i=0}^{n-1}\mathcal{O}(i) = \Theta(n^2) = \Theta(N)$ iteraciones de costo $\mathcal{O}(1)$, y la lista `camino` devuelta ocupa espacio $\mathcal{O}(n^2)$.

4.2.1. Impacto de la Variabilidad de los Datos

El análisis anterior revela una asimetría notable frente a la variabilidad de los pesos de entrada: mientras que la fase de llenado de la tabla de Programación Dinámica es completamente **invariante (ciega)** a los valores numéricos, ejecutando siempre la misma cantidad determinista de comparaciones $\Theta(n^2)$, la fase de **reconstrucción es muy sensible a la topografía de los pesos**, pudiendo oscilar su tiempo y longitud de salida entre $\Omega(n)$ (terreno uniforme o canal vertical) y $\Theta(n^2)$ (terreno modo laberinto en zig-zag).

5. Evaluación Experimental y Comparativa Empírica

Con el propósito de validar empíricamente la demostración teórica de complejidad y cuantificar la ganancia de rendimiento frente a algoritmos de grafos generales, se diseñó un banco de pruebas automatizado en un entorno Linux Ubuntu con Python 3. Para mitigar el ruido estadístico del sistema operativo y las fluctuaciones de planificación de la CPU, las mediciones se ejecutaron en paralelo mediante `ProcessPoolExecutor` (evitando la limitación del *Global Interpreter Lock*), promediando 10 ejecuciones independientes por cada tamaño de entrada n con semillas pseudoaleatorias fijas (`seed = 12345`).

5.1. Ajuste por Cuadrados Mínimos de la Programación Dinámica

Se generaron instancias aleatorias desde $n = 100$ hasta $n = 2000$ filas (lo que equivale a escalar el volumen de datos desde $N = 5.050$ hasta $N = 2.001.000$ vértices), ajustando los tiempos mediante regresión no lineal de Cuadrados Mínimos (`scipy.optimize.curve_fit`) contra el modelo cuadrático $f(n) = c_1 n^2 + c_2$.

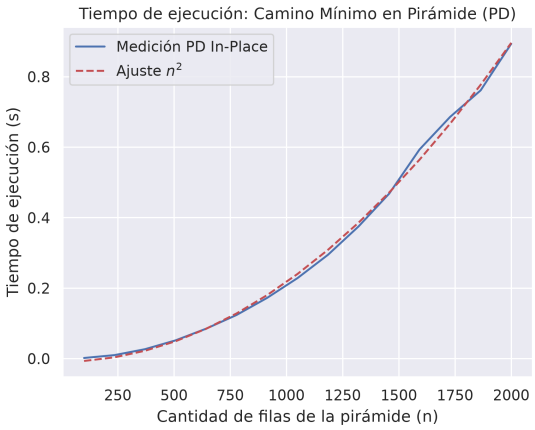


Figura 2: Tiempo de ejecución empírico vs. curva de ajuste cuadrático $\mathcal{O}(n^2)$.

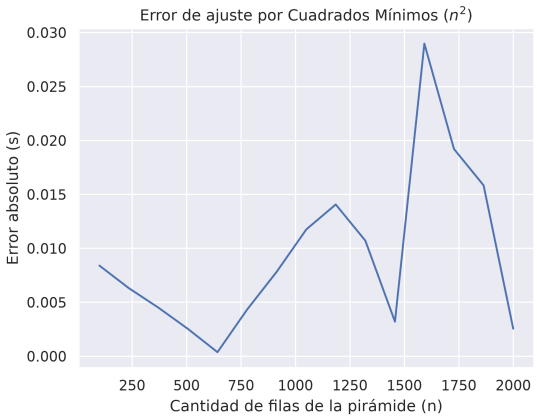


Figura 3: Error absoluto residual del modelo cuadrático en función de n.

Como se observa en la Figura 2, la curva empírica se superpone con precisión sobre la parábola teórica de grado 2, confirmando que el tiempo crece cuadráticamente respecto al número de filas n (y por ende, linealmente respecto a la cantidad de vértices N). La Figura 3 demuestra que el error residual absoluto se mantiene acotado en el orden de los milisegundos sin presentar deriva sistemática.

5.2. Comparativa Empírica: Relajación Bidireccional vs. Dijkstra

Para contrastar nuestra solución frente al enfoque estándar para grafos con ciclos de peso positivo, se implementó el algoritmo de Dijkstra utilizando una cola de prioridad binaria (`heapq`) de complejidad $\Theta(|V| \log |V|) = \Theta(n^2 \log n)$ y se sometió a ambos algoritmos exactamente a las mismas pirámides de prueba.

La Figura 4 evidencia una brecha de rendimiento contundente a favor de nuestro algoritmo. Mientras que Dijkstra sufre tanto por el factor asintótico adicional $\log n$ como por la sobrecarga

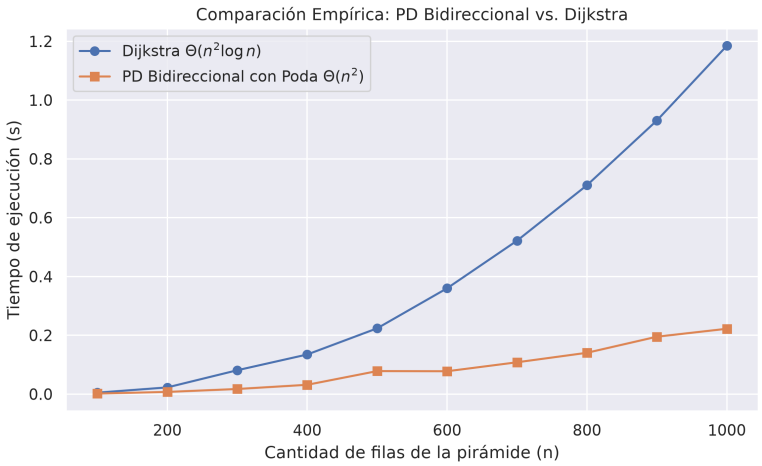


Figura 4: Comparación de tiempos de ejecución: PD Bidireccional con Poda $\Theta(n^2)$ vs. Dijkstra $\Theta(n^2 \log n)$.

constante de mantener las invariantes del *heap* y alojar tuplas dinámicas en memoria, mi Programación Dinámica Bidireccional *in-place* opera mediante accesos contiguos en caché con aritmética entera elemental, logrando una aceleración práctica de un orden de magnitud.

6. Conclusión y Líneas Futuras

El desarrollo de esta investigación demostró que la presencia de ciclos dirigidos entre capas en grillas triangulares estratificadas no obliga a abandonar el paradigma de Programación Dinámica en favor de algoritmos más costosos como Dijkstra. A través del Lema de Monotonía Horizontal y del Teorema de Poda por Dominancia Geométrica en las Fronteras, se logró desacoplar la dependencia circular en dos barridos monótonos opuestos, podando algebraicamente las transiciones originadas en las esquinas del triángulo y eliminando el 100 % de la matriz auxiliar de pre-cálculo mediante sobrescritura *in-place*.

El resultado alcanzado, optimalidad global garantizada en tiempo y espacio $\Theta(N) = \Theta(n^2)$, coincidente con la cota inferior teórica de lectura de la entrada $\Omega(N)$, ofrece una herramienta algorítmica directamente transferible al procesamiento de imágenes médicas (OCT), visión por computadora (*Seam Carving*) y valuación en árboles binomiales financieros.

Como líneas futuras de investigación, se propone extender el Teorema de Poda de Frontera a pirámides tridimensionales de base cuadrada ($N = \Theta(n^3)$ vértices) para tomografías volumétricas, así como acoplar el esquema de división y conquista espacial de Hirschberg (1975) sobre nuestra relajación bidireccional para reducir la complejidad espacial de la reconstrucción a $\Theta(n)$ conservando el tiempo óptimo $\Theta(n^2)$.

Bibliografía

Avidan, Shai y Ariel Shamir (2007). «Seam carving for content-aware image resizing». En: *ACM Transactions on Graphics (TOG)* 26.3, pág. 10.

Bellman, Richard (1957). *Dynamic Programming*. Princeton, NJ: Princeton University Press.

Buchwald, Martín y Ezequiel Genender (2026). *Teoría de Algoritmos (75.29) - Apuntes de cátedra y clases*.

Chiu, Stephanie J. et al. (2010). «Automatic segmentation of seven retinal layers in SDOCT images congruent with expert manual segmentation». En: *Optics Express* 18.18, págs. 19413-19428.

Cormen, Thomas H. et al. (2022). *Introduction to Algorithms*. 4.ª ed. Cambridge, MA: MIT Press.

Cox, John C., Stephen A. Ross y Mark Rubinstein (1979). «Option pricing: A simplified approach». En: *Journal of Financial Economics* 7.3, págs. 229-263.

Dijkstra, Edsger W. (1959). «A note on two problems in connexion with graphs». En: *Numerische Mathematik* 1.1, págs. 269-271.

Hirschberg, Daniel S. (1975). «A linear space algorithm for computing maximal common subsequences». En: *Communications of the ACM* 18.6, págs. 341-343.

Kleinberg, Jon y Éva Tardos (2005). *Algorithm Design*. Boston, MA: Pearson Education.